

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\agent-a83eb777e68482055.jsonl`  
- SHA-256: `b593196721042974f8c629aee0777ef2f6af438947d026c0da27cac288cb4e92`  
- Source modified: `2026-06-21T15:10:58+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `subagents`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T15:09:51.721Z)

Adversarially review a small security-hygiene PR (PACKAGING_PLAN.md P0b) in the git worktree C:/Users/avidu/Projects/bhi-wt-p0b. Goal: GET /api/config returns the whole AppConfig unauthenticated; this PR redacts secret-shaped fields so a future credential field can't leak.

Read these files:

- backend/utils/redact.py (new ? redact_secrets + SECRET_KEY_RE)
- backend/main.py around the `get\_config` handler (search "/api/config")
- tests/test_redact.py and the new test in tests/test_api_endpoints.py (test_api_config_redacts_secret_shaped_fields)

Adversarially check, with concrete file:line evidence:

1. OVER-masking: does SECRET_KEY_RE mask any LEGITIMATE AppConfig field? Enumerate the real field names by reading backend/config/models.py (AppConfig + all sub-config classes) and check each against the regex r"(password|passwd|secret|credential|api[_-]?key|access[_-]?token|auth[_-]?token|refresh[_-]?token)". Flag any false positive (e.g. a field containing 'secret'/'token'/'credential' that is NOT a secret).
2. UNDER-masking: is there any realistic secret-shaped field name the regex would MISS (e.g. 'gemini_key', 'bearer', 'private_key', 'sessionId', connection strings embedded in a non-secret-named field)?
3. Does global_config.model_dump(mode="json") actually surface extra='allow' keys (so the injected gemini_api_key test is valid), and is the JSON shape for non-secret fields byte-compatible with the old `return global\_config` (FastAPI jsonable_encoder of the model)? Any field that serializes differently (e.g. enums, paths)?
4. Any way a secret still leaks: nested under a non-secret key, in a list of dicts, as part of a connection URL value, or via the None-guard returning {}?

Return ONLY: a short list of findings each as "[SEVERITY blocker|major|minor|nit] <file:line> ? <issue> ? <concrete fix>", then a one-line overall verdict. Be precise; do not invent issues. If clean, say so.

2. assistant (2026-06-21T15:09:54.622Z)

I'll review this security PR. Let me start by pulling the worktree fresh and reading the relevant files.

3. user (2026-06-21T15:09:55.590Z)

```
1      """Redact secret-shaped values before they leave the process (PACKAGING_PLAN.md P0b).  
2  
3      ``GET /api/config`` returns the whole ``AppConfig`` unauthenticated. Today AppConfig  
stores NO  
4      secrets by design (the Gemini key lives in the environment; ``cf_access_aud`` is a  
public  
5      identifier), so this is DEFENSIVE: if a credential-shaped field is ever added to the  
config, this
```

```

6     masks it in the response and the accompanying test fails loudly rather than leaking it
to any
7     localhost process/browser tab.
8
9     The match list is deliberately NARROW so it never masks legitimate non-secret fields
such as
10     ``edge_auth_enabled`` (a boolean), ``cf_access_aud`` (a public id), or an ML
``max_tokens`` knob.
11     """
12
13     from __future__ import annotations
14
15     import re
16     from typing import Any
17
18     #: Field-name patterns whose VALUE is masked. Narrow on purpose (see module docstring).
19     SECRET_KEY_RE = re.compile(
20         r"(password|passwd|secret|credential|api[_-]?key|access[_-]?token|auth[_-]
]?token|refresh[_-]?token)",
21         re.IGNORECASE,
22     )
23
24     REDACTED = "****REDACTED****"
25
26
27     def redact_secrets(obj: Any) -> Any:
28         """Return a deep copy of ``obj`` with any value under a secret-shaped KEY replaced
by
29         :data:`REDACTED`. Recurses into dicts and lists; scalars and non-secret keys pass
through.
30
31         A secret-named key masks its WHOLE value (even a nested dict/list ? e.g. a
``credentials``
32         block), so a structured secret can't slip through via recursion.
33         """
34         if isinstance(obj, dict):
35             out: dict[Any, Any] = {}
36             for key, value in obj.items():
37                 if isinstance(key, str) and SECRET_KEY_RE.search(key):
38                     out[key] = REDACTED
39                 else:
40                     out[key] = redact_secrets(value)
41             return out
42         if isinstance(obj, (list, tuple)):
43             return [redact_secrets(item) for item in obj]
44         return obj
45

```

4. user (2026-06-21T15:09:56.252Z)

```

1     """Tests for the secret-redaction helper (PACKAGING_PLAN.md P0b)."""
2
3     from backend.utils.redact import REDACTED, redact_secrets
4
5
6     def test_masks_common_secret_field_names():
7         src = {
8             "gemini_api_key": "AIza-xxx",
9             "api_key": "sk-xxx",
10            "access_token": "tok",
11            "auth_token": "tok2",
12            "refresh_token": "tok3",
13            "client_secret": "shh",
14            "password": "hunter2",
15            "passwd": "hunter2",

```

```

16         "db_credential": "pg://u:p@h",
17     }
18     out = redact_secrets(src)
19     assert all(v == REDACTED for v in out.values()), out
20
21
22     def test_does_not_mask_legitimate_non_secret_fields():
23         # The narrow matcher must NOT mask these real config fields / knobs.
24         src = {
25             "edge_auth_enabled": True,
26             "cf_access_aud": "my-app-aud",
27             "max_tokens": 1024,
28             "sport": "badminton",
29             "output_dir": "output",
30             "tokenizer_name": "bert", # 'token' substring but not an auth token field
31         }
32         assert redact_secrets(src) == src
33
34
35     def test_recurses_into_nested_dicts_and_lists():
36         src = {"storage": {"gcs": {"api_key": "k", "bucket": "b"}}, "items": [{"secret":
"s", "n": 1}]}
37         out = redact_secrets(src)
38         assert out["storage"]["gcs"]["api_key"] == REDACTED
39         assert out["storage"]["gcs"]["bucket"] == "b"
40         assert out["items"][0]["secret"] == REDACTED
41         assert out["items"][0]["n"] == 1
42
43
44     def test_secret_named_key_masks_whole_nested_value():
45         # A structured secret under a secret-named key is masked wholesale (no recursion
leak).
46         src = {"credentials": {"user": "u", "pass": "p"}}
47         assert redact_secrets(src) == {"credentials": REDACTED}
48
49
50     def test_scalars_and_non_dict_inputs_pass_through():
51         assert redact_secrets(42) == 42
52         assert redact_secrets("plain") == "plain"
53         assert redact_secrets([1, 2, 3]) == [1, 2, 3]
54
55
56     def test_does_not_mutate_input():
57         src = {"api_key": "k", "nested": {"token_access": "t"}}
58         snapshot = {"api_key": "k", "nested": {"token_access": "t"}}
59         redact_secrets(src)
60         assert src == snapshot
61

```

5. user (2026-06-21T15:09:56.579Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14

```

```

15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
73         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
74         device: str = "cuda" # auto | cuda | mps | cpu

```

```

75         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
76         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79         keep_frames: bool = False
80         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81         min_free_gb: float = 0.0
82         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87         # Tri-state:
88         #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89         #                     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90         #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91         stream_video: Optional[bool] = None
92
93
94     class FusionConfig(BaseModel):
95         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97         Every default reproduces control behaviour: the fusion segmenter persists the
98         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99         true ? the person-cue features, but it does NOT gate the served handoff unless a
100        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102        supplied ? off-court persons (spectators / adjacent courts) are excluded.
103        """
104        # Which trajectory substrate seeds the windows (shuttle stays primary).
105        substrate: Literal["wasb", "tracknet"] = "wasb"
106        # Windowing velocity threshold for the fusion family (the engine reads
107        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109        velocity_threshold: float = 0.02
110        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111        # enabled ? so the default path never imports torch / touches the GPU.
112        cue_flags: dict[str, bool] = Field(default_factory=dict)
113        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115        min_crossings: int = Field(default=0, ge=0)
116        # Served Gate B: when the person cue is on, drop windows with median in-court
players
117        # < this. 0 = OFF.
118        min_players: int = Field(default=0, ge=0)
119        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120        court_polygon: Optional[list[list[float]]] = None
121        # Net line for the person two-sided-motion split (person features only ? the shuttle

```

```

122     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123     net_axis: Literal["x", "y"] = "y"
124     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
135         pipeline/detectors/{stub,native_wasb}_runner.py.
136         detector_impl: str = "auto"
137         motion_threshold: float = 0.08
138         min_segment_duration: float = 3.0
139         dead_time_merge_gap: float = 2.0
140         chunk_duration: float = 90.0
141         chunk_overlap: float = 10.0
142         detector_model: str = "fasterrcnn_resnet50_fpn"
143         detector_score_threshold: float = 0.5
144         yolo_velocity_threshold: float = 0.15
145         yolo_frame_skip: int = 3
146         yolo_tracker: str = "bytetrack.yaml"
147         yolo_prefetch_workers: int = 2
148         min_action_window_duration: float = 2.0
149         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
150         # = a window
151         # longer than this many seconds is recursively split at its largest internal
152         inactivity gap (?
153         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
154         swallow
155         # several rallies + the dead time between them. Never merges, only cuts (recall
156         can't drop).
157         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
158         Measured n=13
159         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
160         p=0.0005; cap=6
161         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
162         0.651?0.161). ***
163         max_window_duration: float = 6.0
164         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
165         point
166         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
167         inactivity gap
168         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
169         is
170         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
171         bound.
172         # Only cuts (recall can't drop). OFF by default until measured/promoted.
173         window_hard_cap: bool = False
174         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
175         slowly
176         # (picked up / walked) above velocity_thresh, so the window over-extends its END
177         (the measured
178         # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
179         trailing
180         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
181         cuts.
182         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
183         significant
184         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end] 4.96?3.31s (n=13).
185         The W1 cap
186         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md

```

```

"R1". ***
169     window_inactivity_end: float = 1.5
170     inactivity_rally_thresh: float = 0.20
171     inactivity_min_density: float = 0.30
172     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174     # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175     # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176     # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177     # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178     # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179     # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180     window_blob_fallback: bool = False
181     # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182     # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183     # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184     # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185     window_blob_factor: float = 4.0
186     log_yolo_candidates: bool = True
187     store_yolo_candidates: bool = True
188     ai_handoff_padding: float = 2.0
189     ai_max_workers: int = 5
190     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191     # chunks of a high-bitrate (4K) source blow past the provider upload cap
192     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195     # Matches gemini_refine's --clip-scale-width default.
196     ai_chunk_scale_width: int = 1280
197     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202     skip_ai_handoff: bool = False
203     # Optional debug knob: trim every video to the first N seconds before processing.
204     debug_duration: Optional[float] = None
205
206     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208     fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210     @field_validator("default_segmenter")
211     @classmethod
212     def segmenter_must_be_registered(cls, v: str) -> str:
213         # Registry check happens at runtime in main/segmenter selection.
214         # Here we just allow any string for forward compatibility.
215         return v
216
217     @model_validator(mode="after")

```

```

218     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220         # step makes `while t < duration: t += step` loop forever, growing memory before
any
221         # GPU work. Reject the bad config at load time instead.
222         if self.chunk_overlap >= self.chunk_duration:
223             raise ValueError(
224                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
225                 f"({self.chunk_duration}); otherwise chunking never advances."
226             )
227         return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
235         output_dir: str = "output"
236
237
238     class WatermarkConfig(BaseModel):
239         """Branding overlay burned into each highlight clip during the per-clip re-encode
240         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241         (under `stitching.watermark` in config.json) to turn it off. The text is fully
242         configurable. The concat stage stays stream-copy (-c copy)."""
243         enabled: bool = True
244         text: str = "Generated by Khelsutra.guru"
245         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
246         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
247         font: str = "Sans" # fontconfig family used when font_path is empty
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
249         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250         box: bool = True # faint backing box behind the text for legibility
251         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
252         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
253
254
255     class StitchingConfig(BaseModel):
256         begin_padding: float = 0.5
257         end_padding: float = 0.5
258         use_cache: bool = False
259         min_shot_count: int = 1
260         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
261         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
262         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
263         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
264         # local detector over-fires and its false positives are mostly SHORT (motion blips,
265         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
266         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
267         min_rally_duration: float = Field(default=0.0, ge=0.0)
268         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
269
270
271     class LoggingConfig(BaseModel):

```



```

272     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
273     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276     # them to WARNING; set true to restore full chatter for request-flow debugging.
277     verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302         # application's audience tag. NO secrets here ? these are public identifiers.
303         edge_auth_enabled: bool = False
304         cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305         cf_access_aud: str = "" # the CF Access application AUD tag the token must
carry
306         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
308         # OFF ? Cf-Access is a header, not a cookie.)
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
318         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).

```

```

322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + ".x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gs"`` + "bucket/x.mp4" => ``gs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
351
352         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
353         behaves exactly as before this section existed. A profile is opt-in (``config.json``
354         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
365         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).

```

```

370
371     **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
372     today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
373     ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
374     version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
375     inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377     ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
378     wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
379     pricebook + the recording seam; usage emission (the metering hooks) is a separate
step. """
380
381     enabled: bool = False
382     pricebook_version: str = "placeholder-v0" # the seeded $0 version; owner flips to
a real one
383     fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore
market)
384     margin: float = 0.0 # resale markup (?0); 0 = bill at
cost
385
386
387     class AppConfig(BaseModel):
388         """Root configuration object.
389
390         All runtime configuration should be accessed via an instance of this class
391         (or the loader) rather than raw dict lookups.
392         """
393
394         sport: Sport = "badminton"
395         ai_provider: str = "gemini"
396         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
397         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
398         ai_model: str = "gemini-flash-latest"
399
400         validation: ValidationConfig = Field(default_factory=ValidationConfig)
401         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
402         output: OutputConfig = Field(default_factory=OutputConfig)
403         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
404         security: SecurityConfig = Field(default_factory=SecurityConfig)
405         logging: LoggingConfig = Field(default_factory=LoggingConfig)
406         storage: StorageConfig = Field(default_factory=StorageConfig)
407         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
408         billing: BillingConfig = Field(default_factory=BillingConfig)
409
410         # Allow extra keys during transition so existing config.json files do not break.
411         model_config = {
412             "extra": "allow",
413             "validate_assignment": True,
414         }
415
416         # Temporary migration shim for legacy dict-style access (config.get("section",
{})).get(...)).
417         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
418         # attribute access / the typed config. A nested *section* is returned as a plain
dict
419         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
420         # section model instead breaks it (a model has no .get), which bit every validator

```

```

on
421     # real runs. Scalar leaves are returned as-is.
422     def get(self, key: str, default: Any = None) -> Any:
423         _MISSING = object()
424         val: Any = _MISSING
425         if key in type(self).model_fields or hasattr(self, key):
426             val = getattr(self, key, _MISSING)
427         if val is _MISSING:
428             # Legacy convenience: find a leaf key inside any nested section. Sections
are
429             # discovered dynamically from the model fields, so a future top-level
section
430             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
431             for fname in type(self).model_fields:
432                 section = getattr(self, fname, None)
433                 if isinstance(section, BaseModel) and hasattr(section, key):
434                     val = getattr(section, key)
435                     break
436         if val is _MISSING:
437             return default
438         if isinstance(val, BaseModel):
439             return val.model_dump(mode="json")
440         return val
441
442     def __getitem__(self, key: str) -> Any:
443         val = self.get(key)
444         if val is None and key not in type(self).model_fields:
445             # Re-raise like a real dict for unknown top-level
446             raise KeyError(key)
447         return val
448
449
450     # Convenience type alias for callers that want to annotate
451     Config = AppConfig
452

```

6. assistant (2026-06-21T15:09:59.633Z)

Now let me read the `get\_config` handler in main.py and the new API test.

7. user (2026-06-21T15:10:01.669Z)

```

13-# stdlib-only import ? keeps this pre-pydantic line light.
14-from backend.utils.app_home import env_file_to_load, resolve_output_dir
15-
16-load_dotenv(env_file_to_load())
17-
18-# Centralized logging setup for the indexer (hot paths use logger, CLI output uses print for
direct user-facing messages).
19-logging.basicConfig(
20-    level=logging.INFO,
21-    format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
22-    datefmt='%H:%M:%S'
23-)
24-logger = logging.getLogger(__name__)
25-from fastapi import FastAPI, HTTPException, Request
26-from fastapi.middleware.cors import CORSMiddleware
27-from pydantic import BaseModel
28-from typing import List, Dict, Any, Optional
29-
30-from fastapi.staticfiles import StaticFiles
31-from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
32-from backend.pipeline.validators.base import registry
33-from backend.infrastructure.database import Database

```

```

34-from backend.pipeline.segmenters.base import segmenter_registry
35-from backend.pipeline.stitcher.compiler import VideoCompiler
36-from backend.utils.hashing import compute_video_id # canonical file-identity hashing
37-from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
38:from backend.utils.redact import redact_secrets # P0b: mask secret-shaped fields in
/api/config
39-from backend.config import ( # new typed config
40-    load_config as load_app_config,
41-    AppConfig,
42-    StitchingConfig,
43-    enforce_startup_checks,
44-    StartupCheckError,
45-)
46-from backend.results import Failure # standardized error/result contract
47-from backend.reporting import emit_indexer_report
48-from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
49-from backend import billing # M8: per-job cost ledger (USD internal / INR display)
50-from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)
51-
52-app = FastAPI(title="Sports Highlight Indexer API")
53-
54-
55-def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
56-    """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
57-    default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
58-    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
origin."""
59-    src = os.environ if env is None else env
60-    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
61-    origins = [o.strip() for o in raw.split(",") if o.strip()]
62-    return origins or ["*"]
63-
64-
65-
135-    video_id: str
136-    server: Optional[str] = None
137-    receiver: Optional[str] = None
138-    duration_min: Optional[float] = None
139-    event_type: Optional[str] = None
140-
141-class CompileRequest(BaseModel):
142-    video_id: str
143-    segment_ids: List[int]
144-    output_filename: str
145-
146-def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
147-    """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
148-
149-    On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
150-    fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark) and
151-    keep the prior good results intact ? a failed/empty re-run never destroys prior data.
152-    """
153-    if segments:
154-        db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
155-    else:
156-        db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
157-
158-
159-# --- API Endpoints ---
160:@app.get("/api/config")
161-def get_config():
162-    # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped field.
Today
163-    # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public id),

```

```

so this
164- # is byte-identical for the real fields ? but it fails-closed if a secret is ever added
(guarded
165- # by tests/test_redact.py + the /api/config redaction test).
166- if global_config is None:
167-     return {}
168-     return redact_secrets(global_config.model_dump(mode="json"))
169-
170-@app.get("/api/health")
171-def health():
172-     """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
173-     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
174-     confirm the engine is reachable and which detector is wired. Never raises."""
175-     sport = getattr(global_config, "sport", None)
176-     indexing = getattr(global_config, "indexing", None)
177-     return {
178-         "status": "ok",
179-         "sport": sport,
180-         "default_segmenter": getattr(indexing, "default_segmenter", None),
181-     }
182-
183-def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
184-     """The full hash ? validate ? segment ? report pipeline for one video.
185-
186-     This is the former synchronous /api/process body, unchanged in behavior, now run on
187-     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
188-     the sync endpoint used to return (UI compatibility); the per-video observability
189-     report is still emitted in `finally:` and grafted as response["reports"].
190-

```

8. user (2026-06-21T15:10:02.971Z)

```

28-def env(tmp_path, monkeypatch):
29-     db = Database(str(tmp_path / "t.db"))
30-     cfg = AppConfig()
31-     monkeypatch.setattr(m, "db_instance", db)
32-     monkeypatch.setattr(m, "global_config", cfg)
33-     monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
34-     client = TestClient(m.app)
35-     return client, db, tmp_path, monkeypatch
36-
37-
38-def _video(tmp_path, name="m.mp4"):
39-     p = tmp_path / name
40-     p.write_bytes(b"not-a-real-video-but-hashable")
41-     return p
42-
43-
44-def _pass():
45-     return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
46-
47-
48-def _fail():
49-     return [ValidationResult(validator_name="blur", passed=False, message="blurry",
50-                             details={})]
51-
52-class _FakeSeg:
53-     def __init__(self, db, cfg):
54-         self.db = db
55-
56-     def process_video(self, video_id, video_path, *a, **k):
57-         sid = self.db.add_play_segment(video_id, 0.0, 5.0)
58-         return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
59-                 {}]}

```

```

59-
60-
61-def test_api_config_ok(env):
62-     client = env[0]
63-     r = client.get("/api/config")
64-     assert r.status_code == 200
65-     assert r.json()["sport"] == "badminton"
66-
67-
68-def test_api_config_redacts_secret_shaped_fields(env):
69-     """P0b: the unauthenticated /api/config must never serialize a credential-shaped value.
70-     AppConfig stores no secrets today, so inject one via its extra='allow' tolerance and
71-     assert
72-     it is masked while non-secret fields are served unchanged."""
73-     client, db, tmp_path, mp = env
74-     mp.setattr(m, "global_config", AppConfig(gemini_api_key="SECRET-LEAK-123"))
75-     r = client.get("/api/config")
76-     assert r.status_code == 200
77-     assert "SECRET-LEAK-123" not in r.text # the secret never crosses the wire
78-     assert r.json()["gemini_api_key"] == "***REDACTED***"
79-     assert r.json()["sport"] == "badminton" # non-secret fields still served
80-
81-def test_process_success_with_mocked_segementer(env):
82-     """#16: submit returns 202 + job_id; the (inline) job carries the old response."""
83-     client, db, tmp_path, mp = env
84-     vid_file = _video(tmp_path)
85-     mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
86-     mp.setattr(m.segementer_registry, "get", lambda name: _FakeSeg)
87-     r = client.post("\api\process", json={"video_path": str(vid_file), "segementer": "x"})
88-     assert r.status_code == 202
89-     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
90-     assert job["status"] == "done"
91-     assert job["result"]["status"] == "success"
92-     assert len(job["result"]["play_segments"]) == 1
93-
94-
95-def test_process_validation_failure_preserves_prior_segments(env):
96-     client, db, tmp_path, mp = env
97-     vid_file = _video(tmp_path)
98-     vid = m.compute_video_id(str(vid_file))
99-     db.add_video(vid, str(vid_file), "processed", [])
100-     db.add_play_segment(vid, 1.0, 2.0) # prior good result
101-     mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
102-     r = client.post("\api\process", json={"video_path": str(vid_file)})
103-     assert r.status_code == 202
104-     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
105-     assert job["status"] == "done" # worker finished; the PIPELINE verdict failed
106-     assert job["result"]["status"] == "failed"
107-     # PR-C: a failed re-validation must NOT cascade-delete prior segments.
108-     assert len(db.query_play_segments(vid, {})) == 1

```

9. assistant (2026-06-21T15:10:06.846Z)

Now I have all the code. Let me enumerate every AppConfig field against the regex. Let me verify a few edge cases empirically ? particularly whether `model\_dump(mode="json")` surfaces extra keys and whether enum/path fields serialize the same as the old `return global\_config`. Let me check the regex behavior on each field name and confirm the FastAPI serialization equivalence.

10. user (2026-06-21T15:10:18.433Z)

MATCHED fields: []

11. user (2026-06-21T15:10:22.964Z)

```
extra key present in model_dump: True
extra key present in jsonable_encoder: True
byte-equal (non-secret shape): True
```

12. assistant (2026-06-21T15:10:28.775Z)

Both key findings confirmed empirically:

- The regex matches `**zero**` legitimate AppConfig fields (no over-masking).
- ``model_dump(mode="json")`` surfaces the ``extra='allow'`` injected key (test is valid), and the non-secret shape is byte-equal to the old ``jsonable_encoder(global_config)`` path.

Let me double-check one nuance: whether FastAPI's response serialization of the dict (with a ``datetime`/`Path`` anywhere) could differ, and confirm there are no ``Path`/`datetime`/`enum`` typed fields. All config fields are str/int/float/bool/list/dict/Literal ? ``mode="json"`` handles Literals as their str value, identical to `jsonable_encoder`. Confirmed above by byte-equality.

Now the adversarial under-masking analysis. Let me verify the specific names the prompt flagged.

13. user (2026-06-21T15:10:32.606Z)

```
MATCH  gemini_api_key
MATCH  geminiApiKey
MISS   gemini_key
MISS   api_token
MISS   bearer
MISS   bearer_token
MISS   private_key
MISS   privatekey
MISS   sessionId
MISS   session_token
MISS   token
MISS   access_key
MATCH  aws_secret_access_key
MISS   aws_access_key_id
MATCH  client_secret
MISS   signing_key
MISS   jwt
MATCH  jwt_secret
MISS   encryption_key
MISS   pat
MISS   gh_token
MATCH  x_api_key
MISS   authorization
MISS   cookie
MISS   salt
MISS   pin
```

14. assistant (2026-06-21T15:10:58.881Z)

I have everything needed. Here are my findings.

Findings

`**[minor] backend/utils/redact.py:20 ? Under-masking: several realistic secret-shaped names slip the regex.**` The narrow allowlist misses ``private_key``, ``bearer`/`bearer_token``, ``access_key``, ``aws_access_key_id``, ``signing_key`/`encryption_key``, ``session_token``, ``gh_token``, ``authorization``, ``cookie``. Notably ``api_token`` and ``gemini_key`` (a plausible variant of the very example the docstring/test centers on) both MISS ? only ``api_key`/`api-key`/`access_token`` match, not ``*_token`` generally or ``*_key`` generally. This is a defensive future-proofing layer, so a miss only bites if someone later adds such a field AND it isn't caught by the accompanying test. Acceptable for P0b's stated narrow-on-purpose intent, but worth widening. Concrete fix: add alternations ``|token|private[_-]?key|bearer|access[_-]?key|signing[_-]?key|encryption[_-]?key|authorization|cookie`` (accepting that bare ``token`` would also mask ``auth_token`` variants and could catch a future non-secret ``*token*`` field ? tune against the real field list).

****[nit] backend/utils/redact.py:42-43 ? tuples are silently converted to lists.****
`redact\_secrets((1,2))` returns `[1,2]`. Irrelevant for the `/api/config` JSON path (Pydantic `model\_dump(mode="json")` already emits lists), so no live impact; only matters if the helper is reused elsewhere on tuple-bearing data. Concrete fix: none required for this PR; note in the docstring that tuples normalize to lists, or `return type(obj)(...)` if tuple-preservation is ever wanted.

****No over-masking (verified):**** the regex matches ****zero**** of the real AppConfig field names ? enumerated all fields across AppConfig + ValidationConfig/relevance, TrackNet, WasbIndex, Fusion, Indexing, Output, Watermark, Stitching, Logging, Security, Storage, Deployment, Billing. In particular `cf\_access\_aud` (public id), `edge\_auth\_enabled` (bool), `auth`-adjacent names, `access\_\*` (e.g. none present), `\*\_token` (none present), `credential`/`secret` substrings (none present), and `\*\_key` knobs (none present) all pass through unmasked. The `cf\_access\_aud` field contains "access" but not "access_token", so it correctly does NOT match ? no false positive.

****Serialization equivalence (verified):**** `global\_config.model\_dump(mode="json")` is byte-equal to the old `return global\_config` path (`jsonable\_encoder(model)`) for every non-secret field ? confirmed by direct comparison. All config fields are str/int/float/bool/list/dict/`Literal`; `Literal` values serialize as their plain string in both paths, and there are no `Path`/`datetime`/`Enum`/`set` fields that would diverge. So the response shape for real fields is unchanged.

****Test validity (verified):**** `AppConfig(gemini\_api\_key=...)` is accepted via `model\_config extra="allow"` (models.py:411), and `model\_dump(mode="json")` surfaces the extra key ? so the injected `gemini\_api\_key` is genuinely present pre-redaction and the test's `==`
`\*\*\*REDACTED\*\*\*` assertion exercises a real masking, not a no-op.

****Leak-path analysis (clean):**** A secret-named key masks its whole value before recursing (redact.py:37-38), so a structured secret (nested dict/list under e.g. `credentials`) can't escape via recursion ? covered by `test\_secret\_named\_key\_masks\_whole\_nested\_value`. Lists of dicts recurse correctly. The `None`-guard returns `{}` (main.py:166-167), leaking nothing. The one genuine residual gap is values where the secret is in the VALUE not the KEY ? e.g. a connection string in a non-secret-named field like `storage.gcs = {"endpoint\_url": "https://user:pass@host"}` or `input\_root="gs://...@..."`. Redaction is key-name-based only, so such a value would pass through. This matches the documented design (config "stores no secrets? boto3/google auth chains supply credentials", models.py:317) and is out of scope for a key-name redactor; flagging only for completeness, not as a defect of this PR.

****Verdict: LGTM with minor follow-up**** ? correct, no over-masking, serialization byte-compatible, test valid; the only substantive note is the deliberately narrow regex misses some plausible future secret names (`private\_key`, `bearer`, `\*\_token`, `access\_key`), which is consistent with the PR's stated narrow-on-purpose stance but worth widening before a real credential field lands.

(Note: I did not run `git pull`/`run\_all\_tests.py`/ruff/mypy here ? this was a scoped read-only adversarial review of the four named files; full pre-LGTM suite verification per your gate is still owed before sign-off.)